

Native Qwen3-Omni Waveform Runtime and Bounded Audio Context on a 24 GB L4

Implementation Proposal Outline

Systems Architecture and Experimental Plan

August 22, 2026

Implementation sequence at a glance

Stage	Input representation	Historical policy	Purpose and runtime status
A	Full native AuT sequence	Protected prefix plus recent window	Establish the uncompressed, fixed-budget C++/GGUF baseline.
B	Fixed-rate AuT temporal pooling	Protected prefix plus recent window	Establish a deterministic, ASR-free compression baseline at matched token budgets.
C	F16 Whisper-Turbo semantic anchoring	Protected prefix plus recent window	Reproduce VoxZip Stage 1 method on the A100: transcript timestamps, Qwen text embeddings, and aligned AuT pooling.
D	F16 Whisper-Turbo semantic anchoring	VoxZip temporally decayed attention eviction	Reproduce the complete VoxZip method on the A100, documenting any backbone-precision or context-length deviation forced by single-GPU capacity.
E	Fixed-rate AuT temporal pooling	The same temporal eviction used in D	Isolate the value of text-derived semantic anchors by removing Whisper while holding eviction constant.
F	Learned monotonic AuT compactor	Protected prefix plus recent window	Learn transcript-free, chronologically aligned AuT anchors using A100 teacher–student distillation.
G	Learned monotonic AuT compactor	The same temporal eviction used in D and E	Evaluate the intended ASR-free compressed-context method at identical KV bytes.

Whisper precision and residency ladder.

1. Use F16 Whisper-Turbo on the A100 for the unquantized-ASR, method-faithful VoxZip reference in Stages C and D.
2. For the native L4 adaptation, first test Q8_0 Whisper-Turbo. Q8 is preferred because the additional weight cost over Q5_0 is only approximately 0.3 GiB and its numerical perturbation is smaller.
3. Attempt concurrent Q8 residency during listening. Run Whisper in bounded chunks on a separate, lower-priority CUDA stream while AuT and Thinker work remains higher priority.
4. If concurrent residency or compute contention is unacceptable, use sequential Q8 residency: finish AuT, release its weights and workspace, load Whisper into the same arena, construct the anchors, then release Whisper before output generation.
5. Use Q5_0 only if Q8 still exceeds the measured deployment envelope. Apply the same concurrent-then-sequential residency tests to Q5.

6. Treat the F16 A100 result as the exact reference and the Q8/Q5 L4 results as quantized deployment adaptations of the same algorithm.

Project objective

- Execute **Qwen3-Omni-30B-A3B-Instruct** as a native waveform-to-waveform system in one C++ process using GGML and GGUF.
- Fit the complete runtime on one **24 GB NVIDIA L4**; this is a hard deployment constraint.
- Preserve the intelligence-critical Thinker at **Q4_K_M** and the Talker at **Q8.0**.
- Bound accelerator-resident conversational memory independently of interaction length.
- Avoid Python serving and external TTS. Whisper is admitted only as a VoxZip reference baseline; the intended final method remains ASR-free.
- Expose sufficient native runtime state to instrument allocation, routing, attention, and KV behavior and to implement custom bounded-memory policies.

1 Native Qwen3-Omni waveform runtime

1.1 Establish the native Thinker baseline

- Build the current Qwen3-Omni Q4_K_M GGUF path in `llama.cpp`.
- Validate audio input through the existing multimodal projector path before adding output synthesis.
- Record a reproducible baseline for:
 - tensor inventory, names, dimensions, data types, and GGUF metadata;
 - loaded-weight memory by module and backend;
 - CUDA graph workspace and allocator overhead;
 - Thinker KV allocation at each supported context size;
 - peak VRAM during prefill and decode;
 - prefill latency, decode latency, and throughput.
- Separate decimal file size, binary device allocation, and measured CUDA peak memory. Do not infer deployability from GGUF file size alone.
- Preserve fixed reference inputs and Thinker logits for regression testing throughout the port.

1.2 Build an audio-only GGUF package

- Retain the Q8 AuT encoder and every audio projection, normalization, and temporal-encoding tensor required by the Thinker and Talker.
- Remove the vision encoder, vision-only projector tensors, and vision-only runtime paths.
- Preserve sensitive audio projection and normalization tensors at their selected higher precision where conversion requires it.
- Validate the audio-only package against the original multimodal package on identical audio inputs: final AuT and projector representations, Thinker logits, internal generated token sequence, memory, and latency.

1.3 Extend GGUF conversion for speech output

- Convert and package Q8.0 Talker, BF16 MTP, BF16 Code2Wav, speaker/voice-conditioning embeddings, and all metadata required for streaming generation.
- Define independent, versioned tensor namespaces for Thinker, AuT, Talker, MTP, and Code2Wav.

- Verify tensor names, dimensions, strides, data types, checksums, and correspondence with the reference PyTorch weights.
- Keep norms, routers, embeddings, and sensitive output heads at BF16 where quantization offers little benefit or disproportionate risk.

1.4 Port the Talker to GGML

- Implement the 20-layer Talker MoE graph and its independent KV state.
- Implement prompt processing and transfer of model-native conditioning representations from the Thinker.
- Preserve Q8 Talker weights exactly; CPU residency changes placement, not numerical precision.
- Validate dense sublayers, router logits and expert identifiers, expert outputs, layer hidden states, Talker logits, and primary codec predictions against fixed PyTorch references.

1.5 Port the hierarchical MTP loop

- Generate the primary codec value from the Talker.
- Autoregressively generate the remaining residual codebooks through MTP, conditioned on preceding codebooks in the same codec frame.
- Maintain fixed, bounded MTP state for the current codec frame.
- Validate every codebook prediction and intermediate MTP hidden state against the reference implementation.
- Reuse one MTP scratch allocation across residual-codebook substeps.

1.6 Port streaming Code2Wav

- Reconstruct the complete causal Code2Wav graph with GGML CPU operators, adding kernels only where profiling demonstrates a gap.
- Maintain bounded causal left context and produce incremental 24 kHz PCM.
- Resample only at the external audio interface when another sample rate is required.
- Validate waveform reconstruction from fixed codec sequences before integrating generation.
- Measure CPU real-time factor, thread count, memory bandwidth, and interference with pinned expert storage.

1.7 Implement the Thinker–Talker scheduler

- Coordinate incoming chunks, AuT encoding, Thinker prefill/generation, Talker conditioning, MTP codec generation, and Code2Wav streaming.
- Preserve separate state domains for AuT frames, Thinker positions, Talker positions, codec frames, and PCM samples.
- Implement asynchronous Thinker–Talker operation and bounded inter-module queues.
- Support cancellation, cleanup, and phase transitions during barge-in.
- Keep the entire serving path in one native C++ process.

2 Static L4 memory architecture

2.1 Apply the fixed precision policy

- Thinker: Q4_K_M.
- AuT/projector: Q8_0, with sensitive tensors at higher precision where required.
- Talker: Q8_0.

- MTP: BF16.
- Code2Wav: BF16 on CPU.
- Thinker and Talker KV: Q8_0.
- Norms, routers, embeddings, and sensitive heads: BF16 where required.
- Vision tensors: absent.

2.2 Implement phase-overlaid module residency

- Keep the Thinker resident throughout the session.
- During ordinary listening, place AuT in the shared modality arena.
- During VoxZip-reference listening, allow AuT and Whisper to coexist only if measured allocation remains safe.
- During sequential VoxZip execution, release AuT before loading Whisper into the same arena.
- During speaking, release AuT and Whisper and use the arena for the Talker expert-slot pool.
- Keep Code2Wav on CPU throughout output synthesis.
- Maintain CPU-side endpointing and interruption detection while speaking.

2.3 Build a lifetime-managed GGML workspace

- Replace separate maximum-sized workspaces with a lifetime-aware shared arena.
- Reuse temporary allocations across AuT, Whisper when enabled, Thinker, Talker, repeated MTP substeps, and Code2Wav chunks.
- Recycle logits, temporary embeddings, and emitted codec/PCM buffers after their last consumer.
- Tune batch and microbatch sizes for one streaming conversation rather than generic serving.
- Instrument high-water marks for weights, KV, scratch, transfer staging, and allocator overhead separately.

3 Exact Q8 Talker expert residency

3.1 Divide permanent and cacheable Talker weights

- Keep dense attention tensors, routers, embeddings, norms, and codec output heads resident during output generation.
- Keep authoritative Q8 copies of all routed experts in pinned CPU RAM.
- Allocate an initial fixed 1.5 GiB GPU expert-slot pool, then tune it from measured hit-rate and peak-memory curves.
- Never approximate, prune, or substitute a routed expert; a miss changes latency, not weights.

3.2 Implement expert-slot addressing

- Maintain an expert-to-slot mapping independently for each Talker layer.
- Dispatch cache hits directly to resident Q8 tensors.
- On a miss, select a victim, transfer the exact expert asynchronously into a staging slot, wait only for uncovered transfer time, atomically update the mapping, and execute it.
- Track raw transfer time and uncovered stall time separately:

$$T_{\text{stall}} = \max(0, T_{\text{copy}} - T_{\text{overlap}}).$$

3.3 Implement deterministic latency controls

- Use pinned host storage, persistent GPU slots, a dedicated DMA stream, and per-layer active/staging buffers.
- Prefetch experts from current generation state and trace-derived hot sets.
- Begin with fixed-frequency hot sets, LRU, and segmented LRU.
- Collect L4 routing traces and replay them offline to calculate reuse distance, working sets, hit curves, and oracle performance.
- Add predicted future-stall eviction only if deterministic policies leave a meaningful measured gap to the oracle.

3.4 Isolate playback from inference jitter

- Place an initial 320 ms PCM buffer between Code2Wav and the audio interface.
- Track occupancy as

$$B_{t+1} = B_t + T_{\text{generated}} - T_{\text{elapsed}}.$$

- Prefetch the initial Talker hot set before playback begins.
- Keep occupancy above a measured threshold and use bounded clause-boundary pause extension only as an underflow safeguard.
- Measure audible interruptions, underflows, first-audio latency, and 99th-percentile generation real-time factor.

4 Native bounded-KV foundation

4.1 Implement native KV instrumentation

- Begin only after short-context waveform inference is correct.
- Track Thinker positions by conversational block and associate audio positions with original AuT intervals.
- Record per-layer/head KV allocation, attention statistics, boundaries, and eviction decisions without Python serving.
- Distinguish weights, KV, workspace, transfer staging, and allocator overhead.
- Maintain a recoverable CPU ledger containing original audio intervals, AuT ranges, compacted representations, and position metadata.

4.2 Define the fixed Thinker budget

- Begin with 4,096 Q8 KV positions: 512 protected positions, 2,560 exact recent positions, and 1,024 historical positions.
- Make the recent/historical boundary elastic while keeping the total fixed.
- Preserve the protected prefix, current input, and immediately preceding conversational neighborhood.
- Exclude confidently detected silence, ringback, and hold music before inference.
- Preserve wall-clock time and non-neural runtime state outside KV.

4.3 Implement bounded context reconstruction

- Keep recent blocks as exact native representations.
- Generate and retain a compacted candidate for every completed audio block in CPU memory.
- At budget pressure, rebuild at a safe boundary from

[protected prefix; compacted history; exact recent context].

- Re-prefill this bounded sequence to produce ordinary model KV.

- Apply consistent history decisions to Talker conditioning state that spans turns; keep MTP and Code2Wav independently bounded.
- Measure rebuild frequency, amortized prefill cost, and divergence from full context.

5 Staged bounded-context experiments

5.1 Stage A: full AuT with a recent window

- Retain the protected prefix and newest complete blocks that fit.
- Evict the oldest unprotected complete block.
- Use no compression, learned representation, or attention selection.
- Establish the simplest fixed-budget quality, latency, and memory curve.

5.2 Stage B: fixed-rate AuT temporal pooling

- Insert the hook after final AuT projection and before Thinker prefill.
- For $E = [e_1, \dots, e_L]$ and factor c , compute

$$r = \left\lceil \frac{L}{c} \right\rceil, \quad z_j = \frac{1}{|\mathcal{G}_j|} \sum_{k \in \mathcal{G}_j} e_k.$$

- Test $c \in \{2, 4, 8\}$, approximately one vector per 160, 320, and 640 ms at AuT’s 12.5 Hz rate.
- Never pool across speech boundaries, speaker turns, independent events, or streaming discontinuities.
- Compare dense renumbering, first-frame time, and center-frame time for temporal positions.
- Stream with at most $c - 1$ unfinished frames and flush residual groups at boundaries.
- Call this fixed-rate native AuT temporal pooling, not semantic anchoring.
- Implement it directly in GGML/C++; it has no new weights to serialize.

5.3 Stage C: reproduce VoxZip semantic anchoring

- Reproduce the published first stage on the A100 with F16 Whisper-Turbo.
- Attempt the published backbone precision and context length only if the available single A100 can hold them. Otherwise run the same method on the target Q4 backbone and explicitly report the precision/context deviation from the published eight-A100 setup.
- Whisper supplies transcript text, timestamps, and confidence; Qwen supplies AuT representations and its text embedding matrix.
- Retain Whisper segments above the published confidence threshold of 0.3 and map each interval to its AuT range.
- For L_t Qwen transcript tokens, partition L_a AuT vectors into L_t groups and compute

$$\tilde{e}_{a,j} = \frac{1}{|\mathcal{G}_j|} \sum_{k \in \mathcal{G}_j} e_{a,k}.$$

- Fuse pooled audio and corresponding Qwen text embedding:

$$e_{f,j} = \tilde{e}_{a,j} + e_{t,j}.$$

- Preserve background intervals without high-confidence transcripts and concatenate all intervals chronologically.
- Use recent-window retention so Stage C isolates semantic anchoring from eviction.
- Verify the A100 reproduction before adapting it to the L4.

5.4 Native L4 Whisper adaptation for Stages C and D

- Integrate `whisper.cpp` into the native process; no Python server is permitted.
- Attempt Q8_0 before Q5_0. Compare both with F16 on transcripts, timestamps, anchor count, fused-anchor similarity, and downstream output.
- **Concurrent path:**
 1. keep Q8 Whisper resident while listening if peak memory permits;
 2. process bounded chunks on a separate, lower-priority CUDA stream;
 3. prioritize AuT and Thinker work;
 4. finalize Whisper immediately after endpointing;
 5. construct anchors;
 6. release AuT and Whisper before loading Talker expert slots.
- Under concurrency, added latency is approximately

$$T_{\text{added}} = T_{\text{Whisper tail}} + T_{\text{anchor assembly}}.$$

- **Sequential path:**
 1. AuT processes the complete turn and retains projected representations;
 2. endpoint detection closes the turn;
 3. release AuT weights and workspace;
 4. load Q8 Whisper into the shared arena;
 5. transcribe the buffered turn and generate timestamps;
 6. construct VoxZip anchors;
 7. release Whisper;
 8. prefill Thinker and transition to Talker generation.
- Sequential residency yields

$$M_{\text{shared}} = \max(M_{\text{AuT}}, M_{\text{Whisper}}, M_{\text{Talker expert pool}}).$$

- Keep weights in pinned CPU RAM, pre-create reusable state, and use asynchronous DMA to amortize transfer and initialization.
- If Q8 fails both residency paths, repeat with Q5_0. Q5 is a fallback, not reference precision.
- Also benchmark CPU-only Whisper; retain it only if its real-time and post-endpoint latency gates pass without interference.

5.5 Stage D: complete VoxZip reproduction

- Add temporally decayed accumulated-attention eviction to Stage C anchors.
- Reproduce published decay $\gamma = 0.95$ before tuning.
- Decay historical scores before adding current attention contribution, reducing early-token accumulation bias.
- Retain protected positions and select remaining KV entries under a fixed byte budget.
- Reproduce the method on the A100 first, documenting any single-GPU deviation, then validate Q8 and, if required, Q5 L4 adaptations.
- Treat D as the strongest external-ASR reference, not the intended final single-model method.

5.6 Stage E: fixed pooling with the same temporal eviction

- Apply D’s identical retention budget and temporal policy to B’s fixed pooled representations.
- Match initial representation length to VoxZip anchor count or enforce identical total KV bytes.
- Attribute the D–E difference to Whisper-derived semantic anchoring, subject to quantization/runtime controls.
- Establish the strongest train-free, ASR-free native baseline.

5.7 Stage F: learned monotonic AuT compactor

- Implement one selected learned monotonic architecture, not separate learned and monotonic variants.
- For length L and compression c , emit $r = \lceil L/c \rceil$ chronologically ordered vectors.
- Associate query q_j with temporal neighborhood $\mathcal{N}_j$:

$$u_j = \text{Attention}(q_j, E[\mathcal{N}_j]), \quad z_j = \bar{e}_j + \text{MLP}(u_j).$$

- Initialize near fixed pooling, preserving ordered outputs and unambiguous representative timestamps.
- Freeze AuT, Thinker, Talker, MTP, and Code2Wav; train only the compactor initially.
- Distill continuation-token behavior from full-input A100 teacher runs:

$$\mathcal{L}_{\text{KL}} = \sum_t D_{\text{KL}}(p_{\text{full}}(y_t) \| p_{\text{compact}}(y_t)).$$

- Optionally match selected hidden states at common continuation-token positions:

$$\mathcal{L}_h = \sum_{\ell \in \mathcal{S}} \sum_{t \in \text{continuation}} \left\| h_{\ell, t}^{\text{full}} - h_{\ell, t}^{\text{compact}} \right\|_2^2.$$

- Train with $\mathcal{L} = \mathcal{L}_{\text{KL}} + \lambda_h \mathcal{L}_h$.
- Use checkpointing, minimal microbatches, and cached teacher outputs as required by A100 memory.
- Compare fixed and learned methods at identical output lengths and temporal policies.
- Advance only if learned compaction materially improves quality at the same budget.

5.8 Export and execute the learned compactor natively

- Define GGUF metadata for architecture, widths, compression rate, query count, temporal neighborhood, and precision.
- Convert compactor parameters into GGUF tensors traceable by name and checksum to the PyTorch checkpoint.
- Implement the GGML graph at the validated post-projection/pre-Thinker interface.
- Validate embeddings and Thinker logits against PyTorch.
- Measure weight allocation, workspace, latency, and L4 peak memory.
- This export is compactor-specific; all other KV policies still require native C++/GGML execution even without new weights.

5.9 Stage G: learned compactor with temporal eviction

- Combine F’s learned native anchors with the identical decay policy and byte budget used in D and E.
- Compare G versus F for eviction value, G versus E for learned versus fixed transcript-free compaction, and G versus D for native versus transcript-derived anchors.
- Adopt G only if it improves on E without violating L4 memory or latency gates.

6 Evaluation and decision gates

6.1 Representation and memory comparisons

- Compare every stage at identical total KV bytes, not merely nominal positions.
- Report input compression, KV allocation, peak VRAM, prefill/rebuild latency, decode throughput, and shared-arena high-water mark.
- Separate representation compression from retention policy through the A–G ablations.
- Report average quality and long-context behavior across duration buckets.

6.2 Whisper quantization checks

- Compare F16, Q8_0, and Q5_0 on identical audio.
- Measure word error, insertion/deletion, timestamp error, confidence-threshold crossings, transcript-token-count disagreement, anchor similarity, and downstream divergence.
- Prefer Q8 unless peak memory fails. Accept Q5 only within a predeclared downstream tolerance of F16.

6.3 Waveform and interaction gates

- Require measured peak below the 24 GB ceiling with safety headroom; initial target: below 22 GiB under worst-case prefill and synthesis.
- Require 99th-percentile generation real-time factor below one.
- Measure endpointing, first-audio latency, barge-in response, buffer underflows, expert stalls, and uninterrupted-output rate.
- Compare end-to-end semantic behavior and waveform quality against the reference PyTorch system.
- Require Q8 Talker output to match the all-resident Q8 reference apart from documented floating-point nondeterminism.

6.4 Implementation gates

1. Correct short-context native waveform inference precedes bounded-KV work.
2. The static runtime fits before conversational compression is relied upon for survival.
3. Exact Talker expert residency passes fidelity and latency gates before cache-policy research.
4. Stage A establishes the fixed-budget baseline.
5. Stages C and D reproduce VoxZip before replacing its components.
6. Stage E establishes the strongest train-free ASR-free baseline.
7. Stage F advances to GGUF only if it beats B at matched length.
8. Stage G becomes final only if it beats E at matched bytes and remains deployable on L4.

7 Deferred work

- Learned slotted episodic memory, learned memory-access gates, and reinforcement-trained cache actions are deferred.
- They may be revisited after the native runtime, expert residency, fixed pooling, exact VoxZip reproduction, learned compactor, and bounded temporal retention are stable.
- They are not required for the first complete L4 implementation or the core A–G comparison.

Primary references

- Qwen Team, Qwen3-Omni Technical Report.
- Qwen, Qwen3-Omni-30B-A3B-Instruct model and configuration.
- ggml-org, Qwen3-Omni GGUF artifacts.
- VoxZip, Semantic-Anchored Temporal KV Cache Compression for Long-Context Audio Inference.
- ggml-org, whisper.cpp native runtime, CUDA backend, quantization, and model inventory.
- NVIDIA, L4 specifications.